Course 4: Deep Learning

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Course 4: Deep Learning

Fourth session. The three previous ones stayed at the level of "classical" Machine Learning: a feature vector goes in, a decision comes out, and the model has a handful of parameters. Today the model itself becomes the object of study.

One question runs through the whole session: **what does an architecture assume about the data?** The Multi-Layer Perceptron assumes nothing, the Convolutional Network assumes a grid, the Transformer assumes a set of elements that interact. This is the same idea as the *priors* of session 2: choosing a model means choosing an assumption, and here the assumption is drawn into the wiring of the network itself.

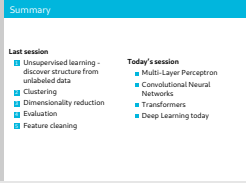
Summary

Last session

1. Unsupervised learning - discover structure from unlabeled data
2. Clustering
3. Dimensionality reduction
4. Evaluation
5. Feature cleaning

Today's session

- Multi-Layer Perceptron
- Convolutional Neural Networks
- Transformers
- Deep Learning today

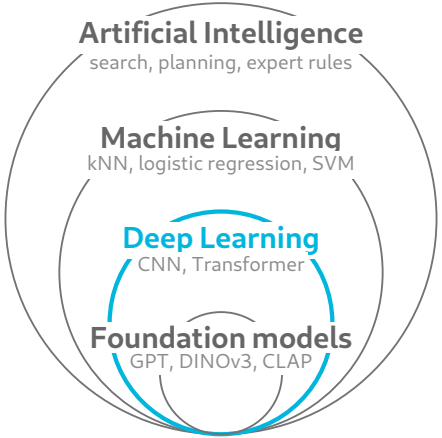


The right column is a progression, not a list of independent topics. It follows chronologically the most important developments in Deep Learning.
The Multi-Layer Perceptron is the vanilla network; it gives the vocabulary: layer, neuron, activation function, loss, backpropagation.
Convolutions then fix its main weakness on images: images are seen as vectors, so local 2D correlations are lost (as in session 3 for MNIST).
Transformers are the architecture behind most recent models, on text and on images alike.
The last part places all of this in the history of the field.

Where the boundary is: today is about **architectures and how they are trained**.

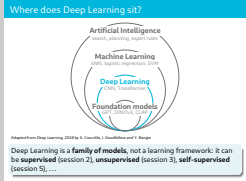
Introducing Transofmers properly, and how one may use very large pre-trained models is the subject of session 5.

Where does Deep Learning sit?



Adapted from *Deep Learning, 2016* by A. Courville, I. Goodfellow and Y. Bengio

Deep Learning is a **family of models**, not a learning framework: it can be **supervised** (session 2), **unsupervised** (session 3), **self-supervised** (session 5), …



This picture already appeared in session 1; it is here only to place today's content.

The sentence that matters is in the block: Deep Learning is often opposed to Machine Learning, or believed to mean supervised classification.

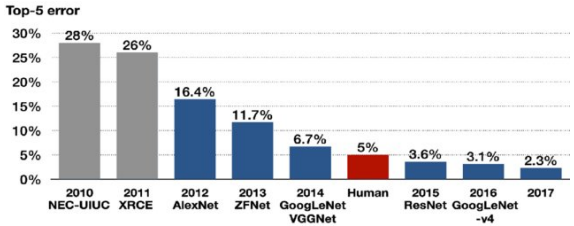
Neither is true: Deep Learning is a class of *models* (neural networks with many layers) and the three frameworks of session 1 (supervised, unsupervised, self-supervised) all apply to them. Today's models are trained in a supervised way, because that is the simplest setting to learn the mechanics in, but nothing in the architectures requires labels, just a (differentiable) cost function!

Deep Learning in a nutshell (1/2)

Tentative definition

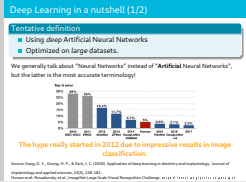
- Using *deep* Artificial Neural Networks
- Optimized on *large* datasets.

We generally talk about "Neural Networks" instead of "**Artificial** Neural Networks", but the latter is the most accurate terminology!



The hype really started in 2012 due to impressive results in image classification.

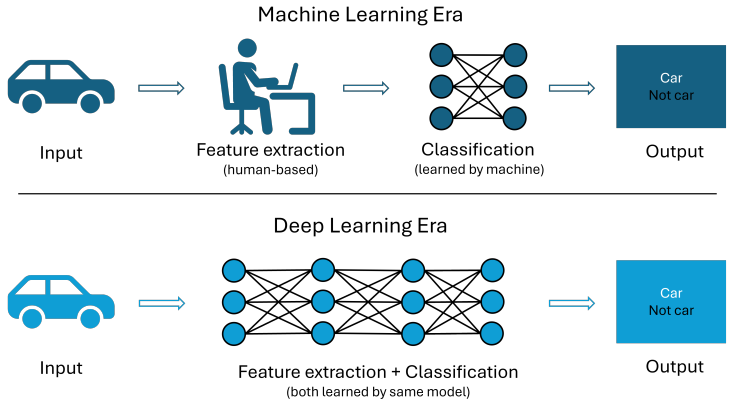
Source: Kang, D. Y., Duong, H. P., & Park, J. C. (2020). Application of deep learning in dentistry and implantology. *Journal of implantology and applied sciences*, 24(3), 148-181.



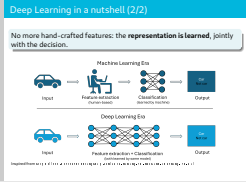
"Deep" refers to the number of stacked layers, and "large" to the size of the dataset. Both matter, and the second is easy to underestimate.
The figure is the error rate on ImageNet, a classification benchmark with a million labelled images, plotted year by year. Until 2011 the winners used hand-designed features and the error decreased slowly, around 26%. In 2012 AlexNet, a convolutional neural network, dropped it to 16%; every winner afterwards was a neural network, and the human error rate (about 5%) was passed in 2015.
That single result is what turned a niche technique into the mainstream of the field (and, indirectly, what put AI in the news). We come back at the end of the CNN section to *why* it happened in 2012 rather than twenty years earlier.
On vocabulary: "Artificial Neural Network" is the accurate term, "Neural Network" the one everybody uses. The word "neuron" is an analogy and nothing more, as a later slide explains.

Deep Learning in a nutshell (2/2)

No more hand-crafted features: the **representation is learned**, jointly with the decision.



Inspired from `https://www.softwaretestinghelp.com/data-mining-vs-machine-learning-vs-ai/`

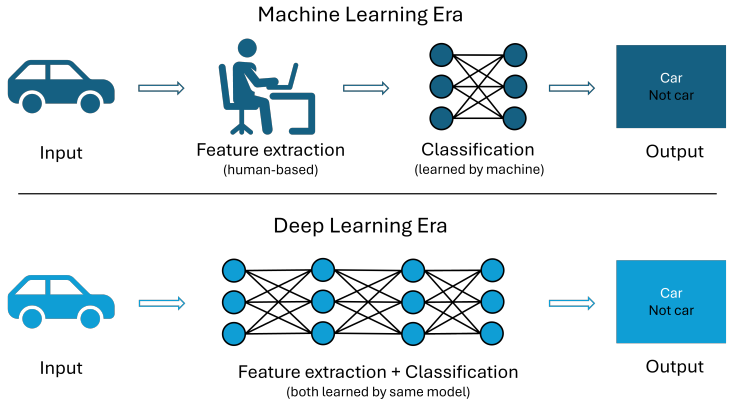


This figure is the real difference between the two pipelines, and it answers the question "what is the vector **x**?" from sessions 2 and 3.
In the classical pipeline, a human designs the features (edges in an image, a spectrogram for audio, ratios in a table) and a small model then draws a boundary in that hand-made space.
In deep learning the two stages are merged into a single function, trained end-to-end: the features become parameters like any others, learned from the data.

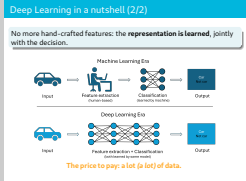
That removes the need for a domain expert to design the representation, and the model may learn any complex and abstract patterns; this is key to the performances! Nonetheless, this is also what makes the method data-hungry: the features have to be inferred from the data. Keep this trade-off in mind for your project, where you have a thousand samples rather than a million (it is precisely why session 5, on reusing pre-trained models, exists).

Deep Learning in a nutshell (2/2)

No more hand-crafted features: the **representation is learned**, jointly with the decision.



The price to pay: a lot *(a lot)* of data.



This figure is the real difference between the two pipelines, and it answers the question “what is the vector **x**?” from sessions 2 and 3.
In the classical pipeline, a human designs the features (edges in an image, a spectrogram for audio, ratios in a table) and a small model then draws a boundary in that hand-made space.
In deep learning the two stages are merged into a single function, trained end-to-end: the features become parameters like any others, learned from the data.

That removes the need for a domain expert to design the representation, and the model may learn any complex and abstract patterns; this is key to the performances! Nonetheless, this is also what makes the method data-hungry: the features have to be inferred from the data. Keep this trade-off in mind for your project, where you have a thousand samples rather than a million (it is precisely why session 5, on reusing pre-trained models, exists).

Outline

1 Multi-layer Perceptron

2 Convolutional Neural Networks (CNN)

3 Transformers

4 Deep Learning today

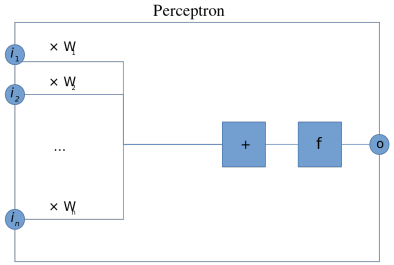
5 Lab session

Outline
1 Multi-layer Perceptron
2 Convolutional Neural Networks (CNN)
3 Transformers
4 Deep Learning today
5 Lab session

Perceptron

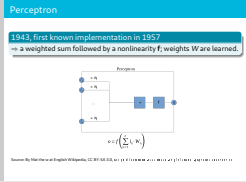
1943, first known implementation in 1957

⇒ a weighted sum followed by a nonlinearity **f**; weights W are learned.



$$o = f\left(\sum_{k=1}^{n} i_k \cdot W_k\right)$$

Source: By Mat the w at English Wikipedia, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=23766733



Take-home message: Neural networks are not a recent invention. The AI timeline of session 1 made the same point: the building blocks were in place decades before the results (stochastic gradient descent in 1952, backpropagation in 1986, ...). What changed after 2010 lies elsewhere, and we come back to it later.

Mathematically, a Perceptron is a weighted sum (a scalar product) followed by a nonlinear function f. Everything in this session (and in deep learning in general) is built by repeating that one operation.

Perceptron

1943, first known implementation in 1957

⇒ a weighted sum followed by a nonlinearity **f**; weights W are learned.

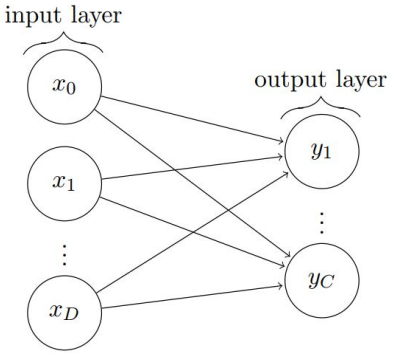
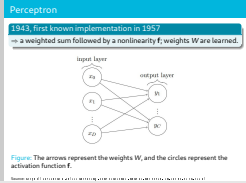


Figure: The arrows represent the weights W, and the circles represent the activation function **f**.



Take-home message: Neural networks are not a recent invention. The AI timeline of session 1 made the same point: the building blocks were in place decades before the results (stochastic gradient descent in 1952, backpropagation in 1986, ...). What changed after 2010 lies elsewhere, and we come back to it later.

Mathematically, a Perceptron is a weighted sum (a scalar product) followed by a nonlinear function f. Everything in this session (and in deep learning in general) is built by repeating that one operation.

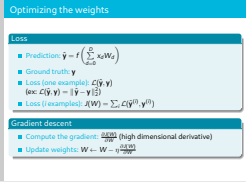
Optimizing the weights

Loss

- Prediction: $\hat{\mathbf{y}} = f\left(\sum_{d=0}^{D} x_d W_d\right)$
- Ground truth: $\mathbf{y}$
- Loss (one example): $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
  (ex: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \| \hat{\mathbf{y}} - \mathbf{y} \|_2^2$)
- Loss (i examples): $J(W) = \sum_i \mathcal{L}(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$

Gradient descent

- Compute the gradient: $\frac{\partial J(W)}{\partial W}$ (high dimensional derivative)
- Update weights: $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$



The adaptation of the formalism from sessions 1 and 2, applied to a single Perceptron. The training machinery does not change when the model becomes deep, only the computation of the gradient does.

The **learning rate** η is a very important hyperparameter, as demonstrated by the visualizations in the next slide.

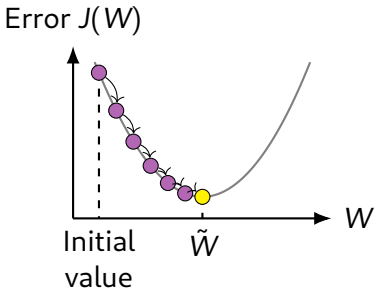
Gradient descent

Intuition behind the gradient descent

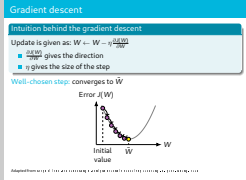
Update is given as: $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

- $\frac{\partial J(W)}{\partial W}$ gives the direction
- η gives the size of the step

Well-chosen step: converges to $\tilde{W}$



Adapted from https://tex.stackexchange.com/questions/561921/replicating-a-plot-using-tikz



The gradient points in the direction in which the error *increases*, so we move in the opposite direction. η sets the length of the step.
The three overlays are the whole slide. With a well-chosen step, the algorithm reaches the bottom in a few iterations. Too small, and it still converges, but you run out of patience (and of computing time). Too large, and it jumps over the minimum and bounces from one side to the other (so the loss may increase instead of decrease).
Practical rule for the lab: **if your loss diverges or oscillates, divide the learning rate by 10** before changing anything else in your model. Typical values run from 10^{-2} to 10^{-4}. Optimizers such as Adam (in `torch.optim`) adapt the step size per parameter, which makes them much less sensitive, and thus very popular.

One caveat about the drawing: it is a one-dimensional cartoon of a landscape that really has millions of dimensions, with plateaus, valleys and many local minima. So in practice, the problem is way more difficult!

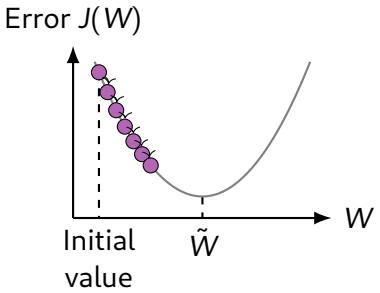
Gradient descent

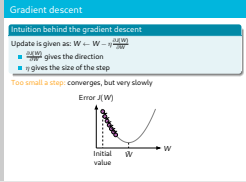
Intuition behind the gradient descent

Update is given as: $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

- $\frac{\partial J(W)}{\partial W}$ gives the direction
- η gives the size of the step

Too small a step: converges, but very slowly





The gradient points in the direction in which the error *increases*, so we move in the opposite direction. η sets the length of the step.
The three overlays are the whole slide. With a well-chosen step, the algorithm reaches the bottom in a few iterations. Too small, and it still converges, but you run out of patience (and of computing time). Too large, and it jumps over the minimum and bounces from one side to the other (so the loss may increase instead of decrease).
Practical rule for the lab: **if your loss diverges or oscillates, divide the learning rate by 10** before changing anything else in your model. Typical values run from 10^{-2} to 10^{-4}. Optimizers such as Adam (in `torch.optim`) adapt the step size per parameter, which makes them much less sensitive, and thus very popular.

One caveat about the drawing: it is a one-dimensional cartoon of a landscape that really has millions of dimensions, with plateaus, valleys and many local minima. So in practice, the problem is way more difficult!

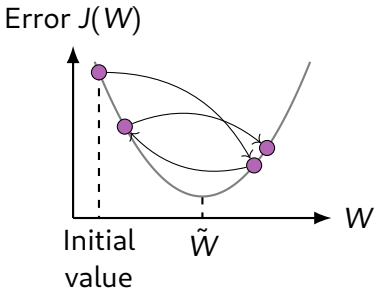
Gradient descent

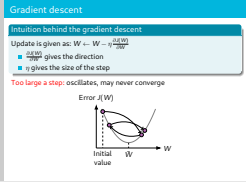
Intuition behind the gradient descent

Update is given as: $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

- $\frac{\partial J(W)}{\partial W}$ gives the direction
- η gives the size of the step

Too large a step: oscillates, may never converge





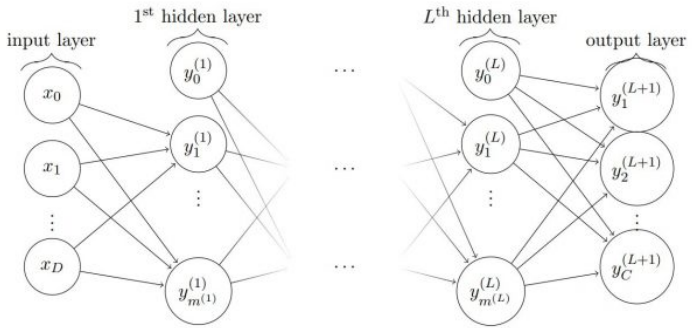
The gradient points in the direction in which the error *increases*, so we move in the opposite direction. η sets the length of the step.
The three overlays are the whole slide. With a well-chosen step, the algorithm reaches the bottom in a few iterations. Too small, and it still converges, but you run out of patience (and of computing time). Too large, and it jumps over the minimum and bounces from one side to the other (so the loss may increase instead of decrease).
Practical rule for the lab: **if your loss diverges or oscillates, divide the learning rate by 10** before changing anything else in your model. Typical values run from 10^{-2} to 10^{-4}. Optimizers such as Adam (in `torch.optim`) adapt the step size per parameter, which makes them much less sensitive, and thus very popular.

One caveat about the drawing: it is a one-dimensional cartoon of a landscape that really has millions of dimensions, with plateaus, valleys and many local minima. So in practice, the problem is way more difficult!

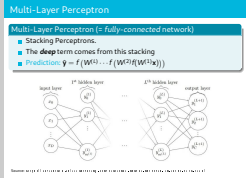
Multi-Layer Perceptron

Multi-Layer Perceptron (= *fully-connected* network)

- Stacking Perceptrons.
- The ***deep*** term comes from this stacking
- Prediction: $\hat{\mathbf{y}} = f\left(W^{(L)} \cdots f\left(W^{(2)} f(W^{(1)}\mathbf{x})\right)\right)$



Source: `https://davidstutz.de/illustrating-convolutional-neural-networks-in-latex-with-tikz/`

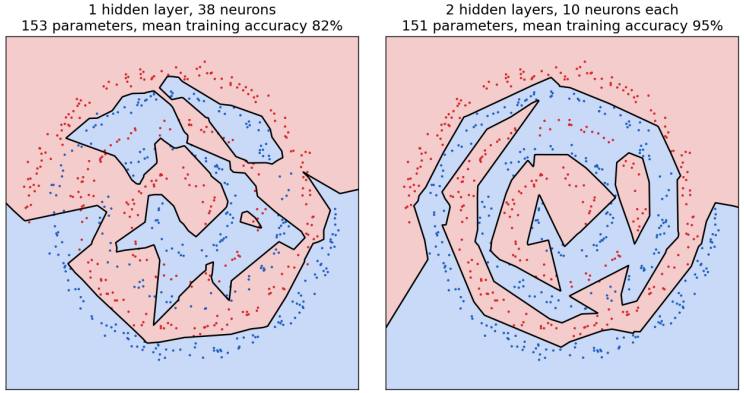


"Fully connected" is the key term, and it is visible on the figure: **every** neuron of a layer is connected to **every** neuron of the next one. Nothing in this wiring says that two inputs are neighbours, or that the input is an image rather than a sound or a table. That is both the generality of the MLP and, two slides from now, its weakness.

In practice, the operation also contains the biases, that are added to the output of each layer. The biases are left out of the equation to keep it readable, but every layer has them.

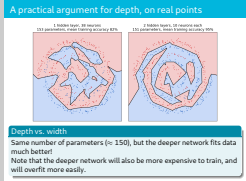
A question that always comes up: how many layers, how many neurons? In theory, a network with two layers (i.e., one "hidden" layer), wide enough, can already approximate any reasonable function (the universal approximation theorem). But it is a statement about *existence*, not about *learnability*: it says nothing about how much data or time would be needed. In practice, depth is what works, and the sizes are found empirically.

A practical argument for depth, on real points



Depth vs. width

Same number of parameters (≈ 150), but the deeper network fits data much better!
Note that the deeper network will also be more expensive to train, and will overfit more easily.



The data are 2D points from two interleaved spirals (red and blue), and the colors in the background show the class each network predicts. Both networks have almost the same number of parameters ($2 \times 38 + 38$ for the hidden layer, plus $38 + 1$ for the output gives 153; $30 + 110 + 11$ gives 151). The only difference is how those parameters are arranged.
With one wide layer, the boundary is a patchwork of pieces placed side by side, and it misses the spiral structure. With two layers, the second layer *reuses and combines* the features built by the first one, so the network can follow the shape of the spiral with the same budget. In other words, for a given number of parameters, depth makes a better use of them than width.

Be careful with what is measured: this is a **training** accuracy. It shows that the deeper network has more expressive power, not that it generalizes better. Its boundary also has odd shapes in empty regions (the triangles in the center), a first sign of overfitting.

Vocabulary

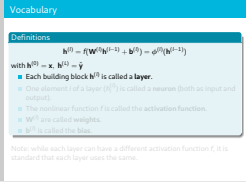
Definitions

$$\mathbf{h}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l-1)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f, it is standard that each layer uses the same.



Important terminology, that you will also find in PyTorch: an `nn.Linear` holds exactly one **W** and one **b**.
The intermediate activations are written **h**, for *hidden*. The output of the last layer, $\mathbf{h}^{(L)}$, is the prediction $\hat{\mathbf{y}}$.

A useful order of magnitude: a layer with n inputs and m outputs has $n \times m$ weights and m biases. So in practice, the number of parameters in a neural network can grow very quickly with the number of layers and neurons.

Vocabulary

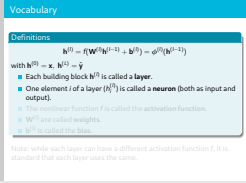
Definitions

$$\mathbf{h}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l-1)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f, it is standard that each layer uses the same.



Important terminology, that you will also find in PyTorch: an `nn.Linear` holds exactly one **W** and one **b**.
The intermediate activations are written **h**, for *hidden*. The output of the last layer, $\mathbf{h}^{(L)}$, is the prediction $\hat{\mathbf{y}}$.

A useful order of magnitude: a layer with n inputs and m outputs has $n \times m$ weights and m biases. So in practice, the number of parameters in a neural network can grow very quickly with the number of layers and neurons.

Vocabulary

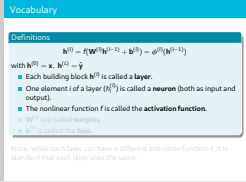
Definitions

$$\mathbf{h}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l-1)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f, it is standard that each layer uses the same.



Important terminology, that you will also find in PyTorch: an `nn.Linear` holds exactly one **W** and one **b**.
The intermediate activations are written **h**, for *hidden*. The output of the last layer, $\mathbf{h}^{(L)}$, is the prediction $\hat{\mathbf{y}}$.

A useful order of magnitude: a layer with n inputs and m outputs has $n \times m$ weights and m biases. So in practice, the number of parameters in a neural network can grow very quickly with the number of layers and neurons.

Vocabulary

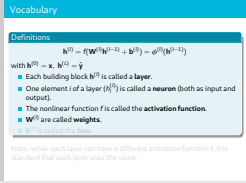
Definitions

$$\mathbf{h}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l-1)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f, it is standard that each layer uses the same.



Important terminology, that you will also find in PyTorch: an `nn.Linear` holds exactly one **W** and one **b**.
The intermediate activations are written **h**, for *hidden*. The output of the last layer, $\mathbf{h}^{(L)}$, is the prediction $\hat{\mathbf{y}}$.

A useful order of magnitude: a layer with n inputs and m outputs has $n \times m$ weights and m biases. So in practice, the number of parameters in a neural network can grow very quickly with the number of layers and neurons.

Vocabulary

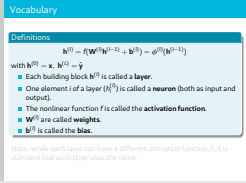
Definitions

$$\mathbf{h}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l-1)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

Note: while each layer can have a different activation function f, it is standard that each layer uses the same.



Important terminology, that you will also find in PyTorch: an `nn.Linear` holds exactly one **W** and one **b**.
The intermediate activations are written **h**, for *hidden*. The output of the last layer, $\mathbf{h}^{(L)}$, is the prediction $\hat{\mathbf{y}}$.

A useful order of magnitude: a layer with n inputs and m outputs has $n \times m$ weights and m biases. So in practice, the number of parameters in a neural network can grow very quickly with the number of layers and neurons.

Vocabulary

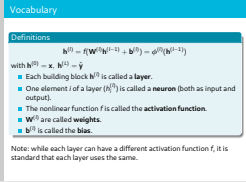
Definitions

$$\mathbf{h}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) = \phi^{(l)}(\mathbf{h}^{(l-1)})$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{h}^{(L)} = \hat{\mathbf{y}}$

- Each building block $\mathbf{h}^{(l)}$ is called a **layer**.
- One element i of a layer ($h_i^{(l)}$) is called a **neuron** (both as input and output).
- The nonlinear function f is called the **activation function**.
- $\mathbf{W}^{(l)}$ are called **weights**.
- $\mathbf{b}^{(l)}$ is called the **bias**.

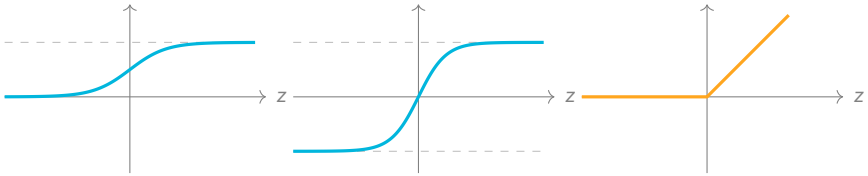
Note: while each layer can have a different activation function f, it is standard that each layer uses the same.



Important terminology, that you will also find in PyTorch: an `nn.Linear` holds exactly one **W** and one **b**.
The intermediate activations are written **h**, for *hidden*. The output of the last layer, $\mathbf{h}^{(L)}$, is the prediction $\hat{\mathbf{y}}$.

A useful order of magnitude: a layer with n inputs and m outputs has $n \times m$ weights and m biases. So in practice, the number of parameters in a neural network can grow very quickly with the number of layers and neurons.

The activation function



sigmoid
$\sigma(z) = 1/(1 + e^{-z})$

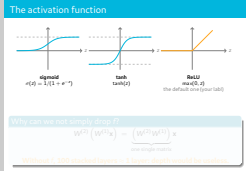
tanh
$\tanh(z)$

ReLU
$\max(0, z)$
the default one (your lab!)

Why can we not simply drop f?

$$W^{(2)}\left(W^{(1)}\mathbf{x}\right) = \underbrace{\left(W^{(2)}W^{(1)}\right)}_{\text{one single matrix}}\mathbf{x}$$

Without f, 100 stacked layers $\approx$ 1 layer: depth would be useless.

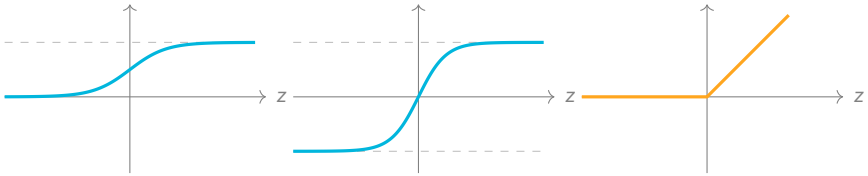


The equation is the whole point: a composition of linear maps is a linear map. Without f, a "deep" network could collapse to a single matrix, that is, to a linear model. **The nonlinearity is important in making depth meaningful.**

About the three curves. Sigmoid and tanh squash their input into a bounded range, and that is one of their defect: far from zero the curve is flat, so its derivative is almost zero. With the chain rule of backpropagation, these small factors multiply along the layers and the gradient *vanishes* (a common problem in deep learning, and one of the reasons deep networks were hard to train before roughly 2010).

ReLU is almost trivial: it zeroes the negative part. Its derivative is exactly 1 on the positive side, it costs one comparison. It has now become the default activation function in many deep learning frameworks, and it is the one you will use in the lab. Sigmoid is still used at the *output* of a binary classifier, where it plays the role of a probability rather than of an activation.

The activation function



sigmoid
$\sigma(z) = 1/(1 + e^{-z})$

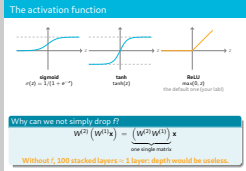
tanh
$\tanh(z)$

ReLU
$\max(0, z)$
the default one (your lab!)

Why can we not simply drop f?

$$W^{(2)}\left(W^{(1)}\mathbf{x}\right) = \underbrace{\left(W^{(2)}W^{(1)}\right)}_{\text{one single matrix}}\mathbf{x}$$

Without f, 100 stacked layers $\approx$ 1 layer: depth would be useless.



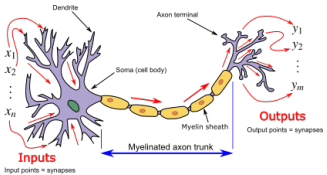
The equation is the whole point: a composition of linear maps is a linear map. Without f, a "deep" network could collapse to a single matrix, that is, to a linear model. **The nonlinearity is important in making depth meaningful.**

About the three curves. Sigmoid and tanh squash their input into a bounded range, and that is one of their defect: far from zero the curve is flat, so its derivative is almost zero. With the chain rule of backpropagation, these small factors multiply along the layers and the gradient *vanishes* (a common problem in deep learning, and one of the reasons deep networks were hard to train before roughly 2010).

ReLU is almost trivial: it zeroes the negative part. Its derivative is exactly 1 on the positive side, it costs one comparison. It has now become the default activation function in many deep learning frameworks, and it is the one you will use in the lab. Sigmoid is still used at the *output* of a binary classifier, where it plays the role of a probability rather than of an activation.

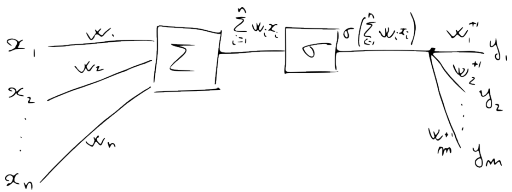
Why is it called Neural Network?

"Neurons" may be seen as **loosely** inspired from the human brain.



Attribution: Egm4313.s12 at English Wikipedia,

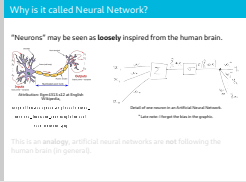
`https://en.wikipedia.org/wiki/Neural_network_(machine_learning)#/media/File:Neuron3.png`



Detail of one neuron in an Artificial Neural Network.

* Late note: I forgot the bias in the graphic.

This is an **analogy**, artificial neural networks are **not** following the human brain (in general).

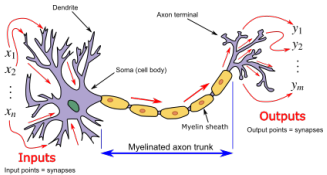


The analogy is historical. In a biological neuron, dendrites collect incoming signals, the cell body integrates them, and beyond a threshold the axon fires. A weighted sum followed by a nonlinearity is a caricature of that mechanism, and this is where the vocabulary comes from.

We insist: **this is an analogy**, because the words may be misleading in public discourse. A biological neuron is a living cell with extremely complex dynamics; an artificial neuron is a scalar product. The structure of the brain is nothing alike one in an artificial neural network. Learning in the brain is not backpropagation. Saying that a network "works like a brain" explains nothing, and it suggests capabilities that the model does not have.

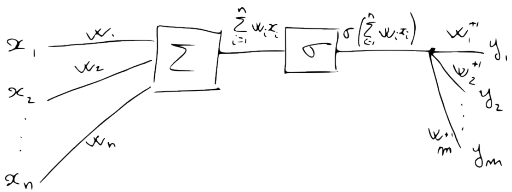
Why is it called Neural Network?

"Neurons" may be seen as **loosely** inspired from the human brain.



Attribution: Egm4313.s12 at English Wikipedia,

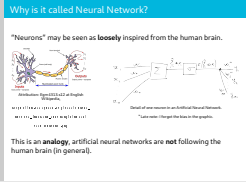
`https://en.wikipedia.org/wiki/Neural_network_(machine_learning)#/media/File:Neuron3.png`



Detail of one neuron in an Artificial Neural Network.

* Late note: I forgot the bias in the graphic.

This is an **analogy**, artificial neural networks are **not** following the human brain (in general).



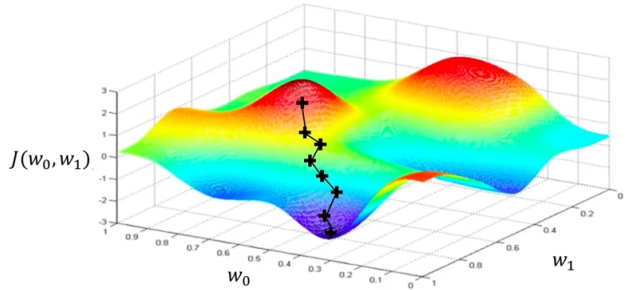
The analogy is historical. In a biological neuron, dendrites collect incoming signals, the cell body integrates them, and beyond a threshold the axon fires. A weighted sum followed by a nonlinearity is a caricature of that mechanism, and this is where the vocabulary comes from.

We insist: **this is an analogy**, because the words may be misleading in public discourse. A biological neuron is a living cell with extremely complex dynamics; an artificial neuron is a scalar product. The structure of the brain is nothing alike one in an artificial neural network. Learning in the brain is not backpropagation. Saying that a network "works like a brain" explains nothing, and it suggests capabilities that the model does not have.

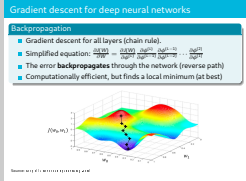
Gradient descent for deep neural networks

Backpropagation

- Gradient descent for all layers (chain rule).
- Simplified equation: $\frac{\partial J(W)}{\partial W} = \frac{\partial J(W)}{\partial \phi^{(L)}} \frac{\partial \phi^{(L)}}{\partial \phi^{(L-1)}} \frac{\partial \phi^{(L-1)}}{\partial \phi^{(L-2)}} \cdots \frac{\partial \phi^{(2)}}{\partial \phi^{(1)}}$
- The error **backpropagates** through the network (reverse path)
- Computationally efficient, but finds a local minimum (at best)



Source: `http://introtodeeplearning.com/`

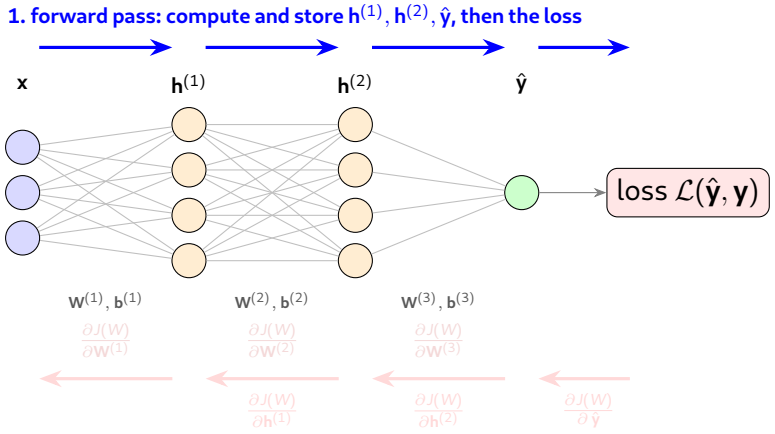


The idea in one sentence: the loss is a composition of functions, so its derivative is a product of derivatives (the chain rule), computed from the output back towards the input; hence *back*propagation. The equation on the slide is a convenient abuse of notation rather than the exact formula; see `https://en.wikipedia.org/wiki/Backpropagation` for the real one.

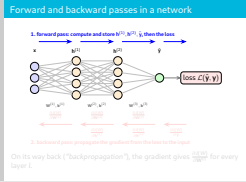
Two consequences worth remembering. It is **cheap**: one backward pass costs roughly as much as one forward pass, whatever the number of parameters, which is why training networks with millions of weights is feasible at all. And it is **local**: it finds a local minimum, with no guarantee about the global one (in very high dimension, this turns out to matter much less in practice than one would fear).

In the lab this is exactly what `loss.backward()` does, with `autograd` building the chain of derivatives automatically. It is also why you must call `optimizer.zero_grad()` first: gradients accumulate by default.

Forward and backward passes in a network



On its way back (*"backpropagation"*), the gradient gives $\frac{\partial J(W)}{\partial \mathbf{W}^{(l)}}$ for every layer l.



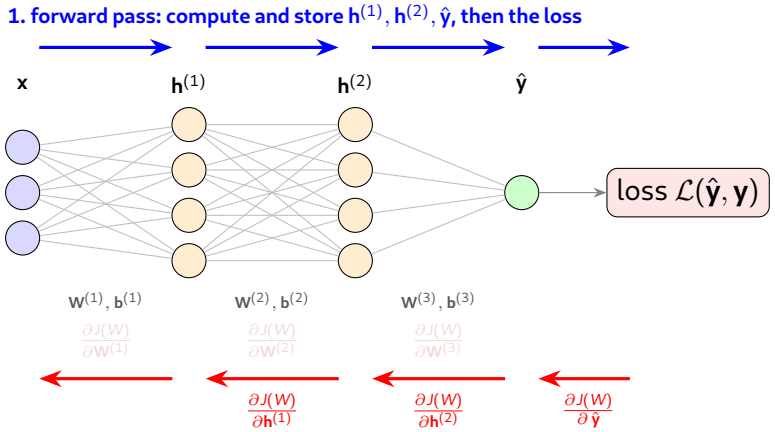
This slide shows the previous equation in action. Training alternates two passes. The **forward pass** computes the prediction layer by layer, and *stores* every intermediate activation $\mathbf{h}^{(l)}$. The **backward pass** starts from the loss and computes, from right to left, the gradient with respect to each activation ($\hat{\mathbf{y}}$, then $\mathbf{h}^{(2)}$, then $\mathbf{h}^{(1)}$): each one is obtained from the previous one by the chain rule.

At each layer, the gradient with respect to the weights (and biases) $\frac{\partial J(W)}{\partial \mathbf{W}^{(l)}}$ only needs two things: the gradient arriving from the right, and the activation $\mathbf{h}^{(l-1)}$ stored during the forward pass. So all the gradients come out of a single backward pass. The price is memory: every activation must be kept until the backward pass, which is often what limits the batch size on a GPU (see batches in the next slide).

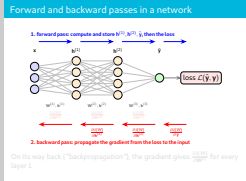
Here, the loss is drawn for one example; $J(W)$ is its sum over many examples, so the same reasoning applies.

In PyTorch: calling `model(x)` is the forward pass, `loss.backward()` is the backward pass, and `optimizer.step()` applies the update $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$.

Forward and backward passes in a network



On its way back (*"backpropagation"*), the gradient gives $\frac{\partial J(W)}{\partial \mathbf{W}^{(l)}}$ for every layer l.



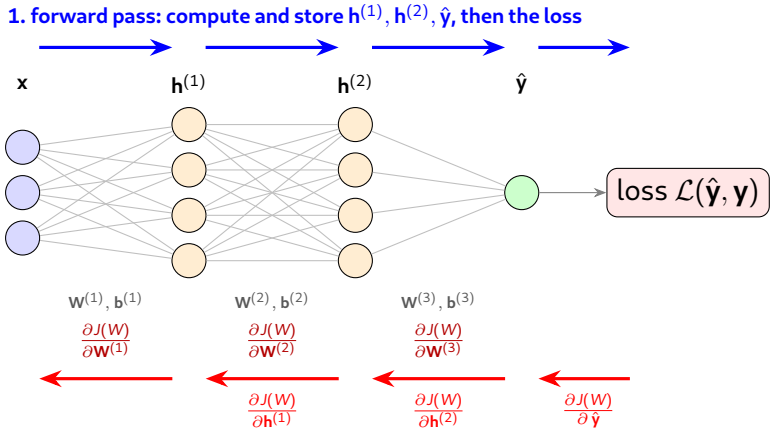
This slide shows the previous equation in action. Training alternates two passes. The **forward pass** computes the prediction layer by layer, and *stores* every intermediate activation $\mathbf{h}^{(l)}$. The **backward pass** starts from the loss and computes, from right to left, the gradient with respect to each activation ($\hat{\mathbf{y}}$, then $\mathbf{h}^{(2)}$, then $\mathbf{h}^{(1)}$): each one is obtained from the previous one by the chain rule.

At each layer, the gradient with respect to the weights (and biases) $\frac{\partial J(W)}{\partial \mathbf{W}^{(l)}}$ only needs two things: the gradient arriving from the right, and the activation $\mathbf{h}^{(l-1)}$ stored during the forward pass. So all the gradients come out of a single backward pass. The price is memory: every activation must be kept until the backward pass, which is often what limits the batch size on a GPU (see batches in the next slide).

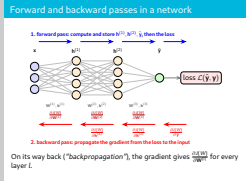
Here, the loss is drawn for one example; $J(W)$ is its sum over many examples, so the same reasoning applies.

In PyTorch: calling `model(x)` is the forward pass, `loss.backward()` is the backward pass, and `optimizer.step()` applies the update $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$.

Forward and backward passes in a network



On its way back (“*backpropagation*”), the gradient gives $\frac{\partial J(W)}{\partial \mathbf{W}^{(l)}}$ for every layer l.



This slide shows the previous equation in action. Training alternates two passes. The **forward pass** computes the prediction layer by layer, and *stores* every intermediate activation $\mathbf{h}^{(l)}$. The **backward pass** starts from the loss and computes, from right to left, the gradient with respect to each activation ($\hat{\mathbf{y}}$, then $\mathbf{h}^{(2)}$, then $\mathbf{h}^{(1)}$): each one is obtained from the previous one by the chain rule.

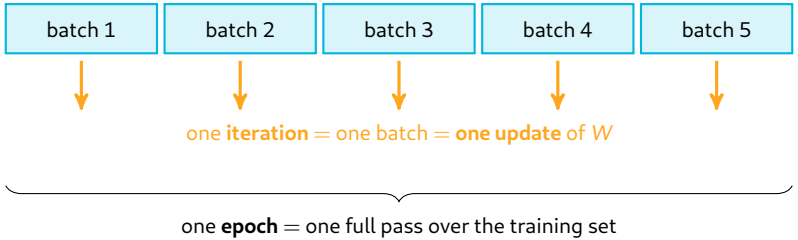
At each layer, the gradient with respect to the weights (and biases) $\frac{\partial J(W)}{\partial \mathbf{W}^{(l)}}$ only needs two things: the gradient arriving from the right, and the activation $\mathbf{h}^{(l-1)}$ stored during the forward pass. So all the gradients come out of a single backward pass. The price is memory: every activation must be kept until the backward pass, which is often what limits the batch size on a GPU (see batches in the next slide).

Here, the loss is drawn for one example; $J(W)$ is its sum over many examples, so the same reasoning applies.

In PyTorch: calling `model(x)` is the forward pass, `loss.backward()` is the backward pass, and `optimizer.step()` applies the update $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$.

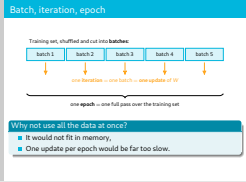
Batch, iteration, epoch

Training set, shuffled and cut into **batches**:



Why not use all the data at once?

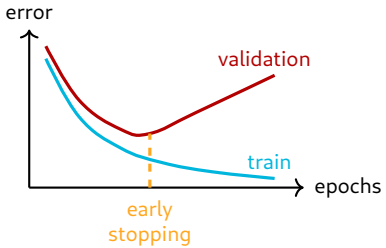
- It would not fit in memory,
- One update per epoch would be far too slow.



Three words that are easy to mix up, and that appear in every training log you will read: a **batch** is a subset of the examples, an **iteration** is one update of the weights computed on one batch, an **epoch** is one full pass over the training set. Do the arithmetic with the lab numbers: 50 000 CIFAR-10 images with `batch_size=8` gives 6 250 iterations per epoch, hence 6 250 updates of the weights. If a single batch contained the whole dataset, one epoch would give one single update. That is one reason for batching; memory is the other one.

A side note: the gradient computed on a batch is a *noisy* estimate of the true gradient, and that is not necessarily a defect. The noise can help escaping bad local minima, and it is one of the reasons why stochastic gradient descent (SGD) is used in place of a full-batch gradient descent.

Overfitting strikes back (Session 2)



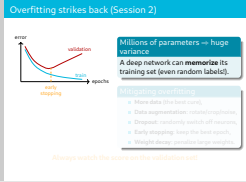
Millions of parameters ⇒ huge variance

A deep network can **memorize** its training set (even random labels!).

Mitigating overfitting

- **More data** (the best cure),
- **Data augmentation**: rotate/crop/noise,
- **Dropout**: randomly switch off neurons,
- **Early stopping**: keep the best epoch,
- **Weight decay**: penalize large weights.

Always watch the score on the validation set!



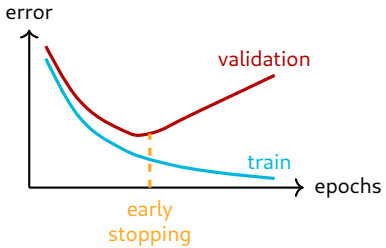
This is the bias/variance slide of session 2, transposed. Nothing conceptual is new; what changes is the scale. A network with more parameters than training examples sits at the extreme "high variance" end. Conceptually, a network can reach perfect training accuracy on a dataset whose labels have been *randomly shuffled*: that is pure memorization. A low training loss means nothing on its own!

Read the figure as the learning curve of session 2, with *epochs* on the horizontal axis: both errors fall, then they separate. The orange line marks where you should stop.

Some tools exist to mitigate overfitting. Data augmentation deserves a word: it is the cheapest of all, and it is a way of *injecting a prior*: a rotated cat is still a cat. That is exactly the "priors and constraints" idea of session 2, and a good way into convolutions, which inject the same kind of knowledge directly into the architecture. It is widely used in practice.

Concretely: keep a validation set aside, print train *and* validation loss at each epoch, and plot them. If you only print the training loss, you cannot see any of this.

Overfitting strikes back (Session 2)



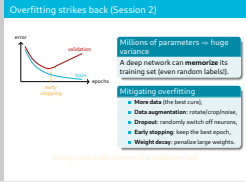
Millions of parameters ⇒ huge variance

A deep network can **memorize** its training set (even random labels!).

Mitigating overfitting

- **More data** (the best cure),
- **Data augmentation**: rotate/crop/noise,
- **Dropout**: randomly switch off neurons,
- **Early stopping**: keep the best epoch,
- **Weight decay**: penalize large weights.

Always watch the score on the validation set!



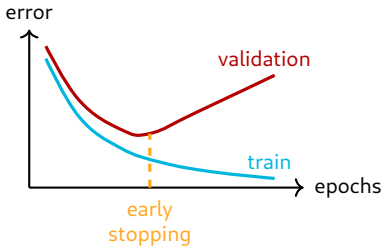
This is the bias/variance slide of session 2, transposed. Nothing conceptual is new; what changes is the scale. A network with more parameters than training examples sits at the extreme "high variance" end. Conceptually, a network can reach perfect training accuracy on a dataset whose labels have been *randomly shuffled*: that is pure memorization. A low training loss means nothing on its own!

Read the figure as the learning curve of session 2, with *epochs* on the horizontal axis: both errors fall, then they separate. The orange line marks where you should stop.

Some tools exist to mitigate overfitting. Data augmentation deserves a word: it is the cheapest of all, and it is a way of *injecting a prior*: a rotated cat is still a cat. That is exactly the "priors and constraints" idea of session 2, and a good way into convolutions, which inject the same kind of knowledge directly into the architecture. It is widely used in practice.

Concretely: keep a validation set aside, print train *and* validation loss at each epoch, and plot them. If you only print the training loss, you cannot see any of this.

Overfitting strikes back (Session 2)



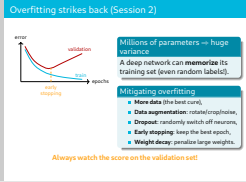
Millions of parameters ⇒ huge variance

A deep network can **memorize** its training set (even random labels!).

Mitigating overfitting

- **More data** (the best cure),
- **Data augmentation**: rotate/crop/noise,
- **Dropout**: randomly switch off neurons,
- **Early stopping**: keep the best epoch,
- **Weight decay**: penalize large weights.

Always watch the score on the validation set!



This is the bias/variance slide of session 2, transposed. Nothing conceptual is new; what changes is the scale. A network with more parameters than training examples sits at the extreme "high variance" end. Conceptually, a network can reach perfect training accuracy on a dataset whose labels have been *randomly shuffled*: that is pure memorization. A low training loss means nothing on its own!

Read the figure as the learning curve of session 2, with *epochs* on the horizontal axis: both errors fall, then they separate. The orange line marks where you should stop.

Some tools exist to mitigate overfitting. Data augmentation deserves a word: it is the cheapest of all, and it is a way of *injecting a prior*: a rotated cat is still a cat. That is exactly the "priors and constraints" idea of session 2, and a good way into convolutions, which inject the same kind of knowledge directly into the architecture. It is widely used in practice.

Concretely: keep a validation set aside, print train *and* validation loss at each epoch, and plot them. If you only print the training loss, you cannot see any of this.

Limits of the MLP

1. Too many parameters

a 224×224 colour image
$d = 150\,528$ values

↓

fully connected,
1 000 neurons
$150\,528 \times 1\,000 \approx$
150 million weights
…for *one* layer

2. No notion of structure

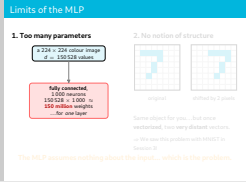
original

shifted by 2 pixels

Same object for you… but once **vectorized**, two **very distant** vectors.

⇒ We saw this problem with MNIST in Session 3!

The MLP assumes nothing about the input… which is the problem.



Two limits, and the next section answers both.

Number of parameters: 150 million weights for a single layer means a model that does not fit on a laptop GPU, that is desperately prone to overfitting (previous slide), and that needs a colossal dataset.

No prior about the structure. This is the loop opened in session 3: when a 28×28 image becomes a vector of 784 numbers, the notion of neighbouring pixels is destroyed, and shifting the digit by two pixels gives a vector far from the original, although to you it is obviously the same digit. An MLP would have to *learn* that invariance from examples, separately for every position. A convolution has it built in.

In a nutshell: the generality of the MLP (i.e. no assumption at all about data) becomes a defect as soon as you know something about your data. A good architecture encodes what you already know, while letting the network learn what is important in data.

Limits of the MLP

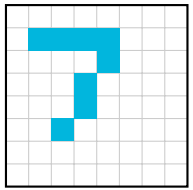
1. Too many parameters

a 224×224 colour image
$d = 150\,528$ values

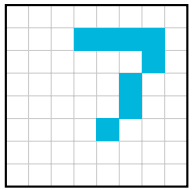
↓

fully connected,
1 000 neurons
$150\,528 \times 1\,000 \approx$
150 million weights
…for *one* layer

2. No notion of structure



original

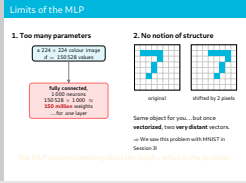


shifted by 2 pixels

Same object for you…but once **vectorized**, two **very distant** vectors.

⇒ We saw this problem with MNIST in Session 3!

The MLP assumes nothing about the input… which is the problem.



Two limits, and the next section answers both.

Number of parameters: 150 million weights for a single layer means a model that does not fit on a laptop GPU, that is desperately prone to overfitting (previous slide), and that needs a colossal dataset.

No prior about the structure. This is the loop opened in session 3: when a 28×28 image becomes a vector of 784 numbers, the notion of neighbouring pixels is destroyed, and shifting the digit by two pixels gives a vector far from the original, although to you it is obviously the same digit. An MLP would have to *learn* that invariance from examples, separately for every position. A convolution has it built in.

In a nutshell: the generality of the MLP (i.e. no assumption at all about data) becomes a defect as soon as you know something about your data. A good architecture encodes what you already know, while letting the network learn what is important in data.

Limits of the MLP

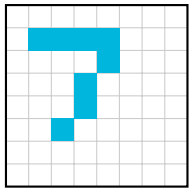
1. Too many parameters

a 224×224 colour image
$d = 150\,528$ values

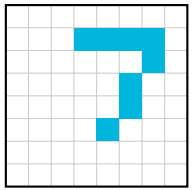
↓

fully connected,
1 000 neurons
$150\,528 \times 1\,000 \approx$
150 million weights
…for *one* layer

2. No notion of structure



original

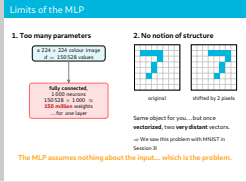


shifted by 2 pixels

Same object for you…but once **vectorized**, two **very distant** vectors.

⇒ We saw this problem with MNIST in Session 3!

The MLP assumes nothing about the input… which is the problem.



Two limits, and the next section answers both.

Number of parameters: 150 million weights for a single layer means a model that does not fit on a laptop GPU, that is desperately prone to overfitting (previous slide), and that needs a colossal dataset.

No prior about the structure. This is the loop opened in session 3: when a 28×28 image becomes a vector of 784 numbers, the notion of neighbouring pixels is destroyed, and shifting the digit by two pixels gives a vector far from the original, although to you it is obviously the same digit. An MLP would have to *learn* that invariance from examples, separately for every position. A convolution has it built in.

In a nutshell: the generality of the MLP (i.e. no assumption at all about data) becomes a defect as soon as you know something about your data. A good architecture encodes what you already know, while letting the network learn what is important in data.

Outline

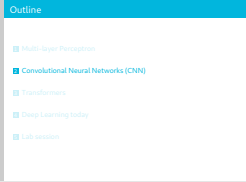
1 Multi-layer Perceptron

2 Convolutional Neural Networks (CNN)

3 Transformers

4 Deep Learning today

5 Lab session

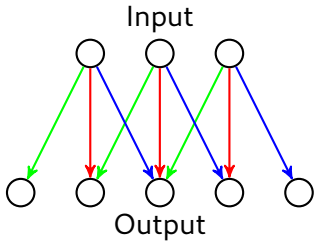


Convolutional Neural Network (1/5): the principle

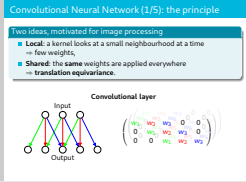
Two ideas, motivated for image processing

- **Local**: a kernel looks at a small neighbourhood at a time
  $\Rightarrow$ few weights,
- **Shared**: the **same** weights are applied everywhere
  $\Rightarrow$ **translation equivariance**.

Convolutional layer



$$\begin{pmatrix} w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \end{pmatrix}$$



Read the drawing as a fully-connected layer from which almost all connections have been **deleted**, and in which the three surviving ones (green, red, blue) are **the same** for every output neuron. The matrix on the right makes it explicit: a full matrix constrained to be very sparse, with only three distinct values repeated along the diagonals. Three weights instead of 3×5.

"Shared" is the important word. The pattern being detected does not depend on the position, so a vertical edge is recognized the same way in the top-left and in the bottom-right corner. This is what an MLP had to learn separately for every position, and it is why *translation equivariance* comes for free: shift the input, and the output shifts identically. Combined with pooling, it becomes an approximate *invariance*.

The pale matrices stacked in the background are the other **channels**, also called filters: a layer learns several kernels in parallel, each looking for a different pattern. In `nn.Conv2d` their number is `out_channels`.

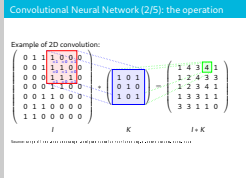
Convolutional Neural Network (2/5): the operation

Example of 2D convolution:

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 3 & 4 & 1 \\ 1 & 2 & 4 & 3 & 3 \\ 1 & 2 & 3 & 4 & 1 \\ 1 & 3 & 3 & 1 & 1 \\ 3 & 3 & 1 & 1 & 0 \end{pmatrix}$$

I $\qquad$ K $\qquad$ $I * K$

Source: `https://tex.stackexchange.com/questions/437007/drawing-a-convolution-with-tikz`



The arithmetic: place the kernel K on a 3×3 patch of the image I, multiply term by term, sum, and write the result in the corresponding cell of the output. Then slide the window and repeat. Follow the three overlays; the blue dotted lines show which patch produces which output value.

Two remarks you will need in the lab. The output is *smaller* than the input: a 7×7 image with a 3×3 kernel gives 5×5. You can compute the output size using the formula $h_{out} = (h_{in} - 2p - k)/s + 1$, where p is the padding, k is the kernel size, and s is the stride. This is helpful but a bit out-of-scope, because we did not introduce the concept of padding and stride yet. You'll see them in the lab.

And an important difference with this example: here the nine numbers of the kernel are given, whereas in a convolutional network they are **parameters**, initialized at random and learned by gradient descent.

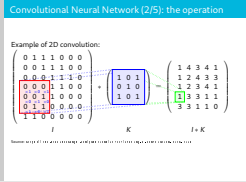
Convolutional Neural Network (2/5): the operation

Example of 2D convolution:

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 3 & 4 & 1 \\ 1 & 2 & 4 & 3 & 3 \\ 1 & 2 & 3 & 4 & 1 \\ 1 & 3 & 3 & 1 & 1 \\ 3 & 3 & 1 & 1 & 0 \end{pmatrix}$$

I $\qquad$ K $\qquad$ $I * K$

Source: `https://tex.stackexchange.com/questions/437007/drawing-a-convolution-with-tikz`



The arithmetic: place the kernel K on a 3×3 patch of the image I, multiply term by term, sum, and write the result in the corresponding cell of the output. Then slide the window and repeat. Follow the three overlays; the blue dotted lines show which patch produces which output value.

Two remarks you will need in the lab. The output is *smaller* than the input: a 7×7 image with a 3×3 kernel gives 5×5. You can compute the output size using the formula $h_{out} = (h_{in} - 2p - k)/s + 1$, where p is the padding, k is the kernel size, and s is the stride. This is helpful but a bit out-of-scope, because we did not introduce the concept of padding and stride yet. You'll see them in the lab.

And an important difference with this example: here the nine numbers of the kernel are given, whereas in a convolutional network they are **parameters**, initialized at random and learned by gradient descent.

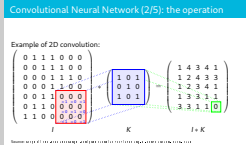
Convolutional Neural Network (2/5): the operation

Example of 2D convolution:

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 3 & 4 & 1 \\ 1 & 2 & 4 & 3 & 3 \\ 1 & 2 & 3 & 4 & 1 \\ 1 & 3 & 3 & 1 & 1 \\ 3 & 3 & 1 & 1 & 0 \end{pmatrix}$$

I K $I * K$

Source: `https://tex.stackexchange.com/questions/437007/drawing-a-convolution-with-tikz`



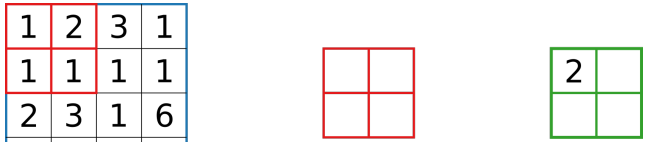
The arithmetic: place the kernel K on a 3×3 patch of the image I, multiply term by term, sum, and write the result in the corresponding cell of the output. Then slide the window and repeat. Follow the three overlays; the blue dotted lines show which patch produces which output value.

Two remarks you will need in the lab. The output is *smaller* than the input: a 7×7 image with a 3×3 kernel gives 5×5. You can compute the output size using the formula $h_{out} = (h_{in} - 2p - k)/s + 1$, where p is the padding, k is the kernel size, and s is the stride. This is helpful but a bit out-of-scope, because we did not introduce the concept of padding and stride yet. You'll see them in the lab.

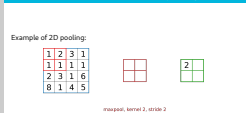
And an important difference with this example: here the nine numbers of the kernel are given, whereas in a convolutional network they are **parameters**, initialized at random and learned by gradient descent.

Convolutional Neural Network (3/5): pooling

Example of 2D pooling:



Convolutional Neural Network (3/5): pooling



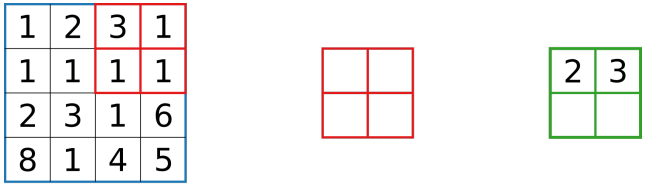
Pooling reduces the resolution: take a 2×2 window and keep only the maximum (or the average). It has **no parameters** to learn.

It has two effects: It divides the size of the maps by a factor (here, four) at each step, which is what keeps the computation affordable as the network gets deeper. And it makes the representation *tolerant to small shifts*: move the pattern by one pixel inside the window and the maximum does not change. Convolution gives equivariance; pooling turns it into an approximate invariance.

The counterpart is that information is lost, deliberately. That is fine for classification ("is there a cat?"), and a problem for tasks that need precise localization, such as segmentation, where other mechanisms are used instead.

Convolutional Neural Network (3/5): pooling

Example of 2D pooling:

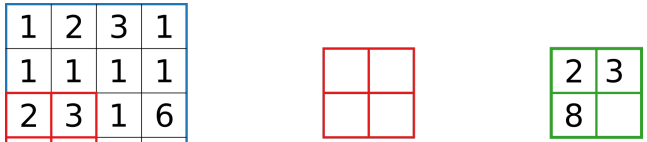


Convolutional Neural Network (3/5): pooling
Example of 2D pooling:
1 2 3 1
1 1 1 1
2 3 1 6
8 1 4 5
2 3
maxpool, kernel 2, stride 2

Pooling reduces the resolution: take a 2×2 window and keep only the maximum (or the average). It has **no parameters** to learn.
It has two effects: It divides the size of the maps by a factor (here, four) at each step, which is what keeps the computation affordable as the network gets deeper. And it makes the representation *tolerant to small shifts*: move the pattern by one pixel inside the window and the maximum does not change. Convolution gives equivariance; pooling turns it into an approximate invariance.
The counterpart is that information is lost, deliberately. That is fine for classification ("is there a cat?"), and a problem for tasks that need precise localization, such as segmentation, where other mechanisms are used instead.

Convolutional Neural Network (3/5): pooling

Example of 2D pooling:

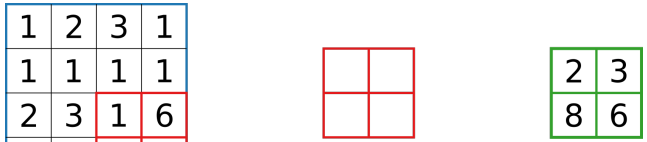


Convolutional Neural Network (3/5): pooling
Example of 2D pooling:
maxpool, kernel 2, stride 2

Pooling reduces the resolution: take a 2×2 window and keep only the maximum (or the average). It has **no parameters** to learn.
It has two effects: It divides the size of the maps by a factor (here, four) at each step, which is what keeps the computation affordable as the network gets deeper. And it makes the representation *tolerant to small shifts*: move the pattern by one pixel inside the window and the maximum does not change. Convolution gives equivariance; pooling turns it into an approximate invariance.
The counterpart is that information is lost, deliberately. That is fine for classification ("is there a cat?"), and a problem for tasks that need precise localization, such as segmentation, where other mechanisms are used instead.

Convolutional Neural Network (3/5): pooling

Example of 2D pooling:



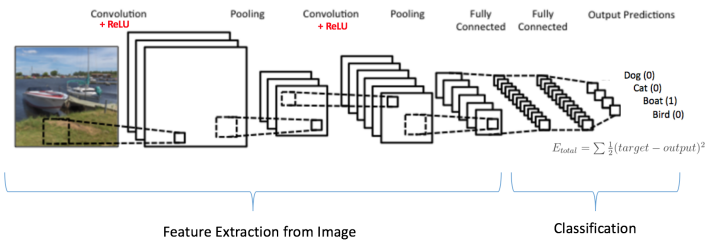
Convolutional Neural Network (3/5): pooling
Example of 2D pooling:
maxpool, kernel 2, stride 2

Pooling reduces the resolution: take a 2×2 window and keep only the maximum (or the average). It has **no parameters** to learn.
It has two effects: It divides the size of the maps by a factor (here, four) at each step, which is what keeps the computation affordable as the network gets deeper. And it makes the representation *tolerant to small shifts*: move the pattern by one pixel inside the window and the maximum does not change. Convolution gives equivariance; pooling turns it into an approximate invariance.
The counterpart is that information is lost, deliberately. That is fine for classification ("is there a cat?"), and a problem for tasks that need precise localization, such as segmentation, where other mechanisms are used instead.

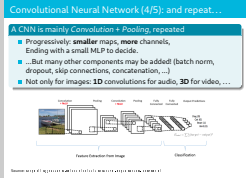
Convolutional Neural Network (4/5): and repeat…

A CNN is mainly *Convolution* + *Pooling*, repeated

- Progressively: **smaller** maps, **more** channels,
  Ending with a small MLP to decide.
- …But many other components may be added! (batch norm, dropout, skip connections, concatenation, …)
- Not only for images: **1D** convolutions for audio, **3D** for video, …



Source: `https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/`



Here is the whole picture. The first layers look at small patches of the image, and they produce a few feature maps. The next layers look at larger patches of the previous maps, and they produce more feature maps. And so on, until the last layer sees the whole image and produces a few numbers that summarize it.

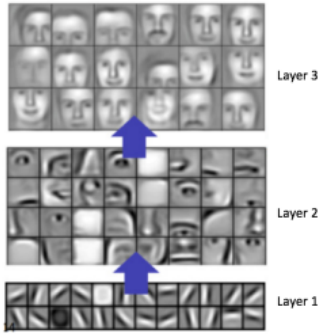
Notice the trade: the spatial size shrinks while the number of channels grows. Intuitively, the network trades "where" for "what". At the end the maps are small enough for a plain MLP to conclude. Everything before it is the learned feature extractor.

Even if convolutions were mainly developed for images, the same principle applies to any data with a regular structure: 1D convolutions for audio, 3D for video, and even 4D for medical imaging.

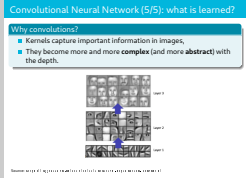
Convolutional Neural Network (5/5): what is learned?

Why convolutions?

- Kernels capture important information in images,
- They become more and more **complex** (and more **abstract**) with the depth.



Source: https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/



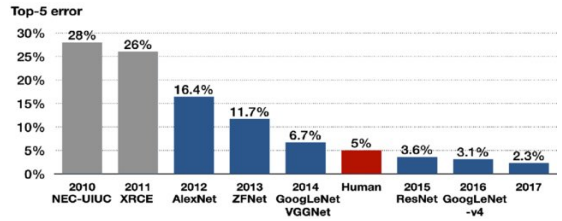
This may be the most important slide bout convolutions: what does "learning the representation" concretely means. The first layers respond to edges, corners and colour gradients, and they look remarkably like the filters an expert would have designed by hand (such as Gabor filters or wavelets). Middle layers respond to assemblies of edges, to textures and patterns. The last ones respond to object parts and whole objects.

So the hierarchy of features that used to be built by hand is here *discovered* by gradient descent, from the labels alone. This is what makes deep learning as performant as it is!

Note though that it may introduce biases: if the training set contains only cats with a certain background, the network may learn to associate that background with the cat class.

Two consequences for later. The first layers are extremely generic, which is why a network pre-trained on ImageNet may be useful on other datasets (such as medical imaging); this is transfer learning. We will discuss this in more detail in session 5.

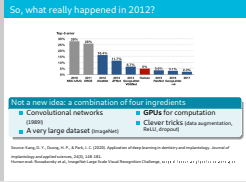
So, what really happened in 2012?



Not a new idea: a combination of four ingredients

- Convolutional networks (1989!)
- A very large dataset (ImageNet)
- **GPUs** for computation
- Clever tricks (data augmentation, ReLU, dropout)

Source: Kang, D. Y., Duong, H. P., & Park, J. C. (2020). Application of deep learning in dentistry and implantology. *Journal of implantology and applied sciences*, 24(3), 148-181.
Human eval: Russakovsky *et al.*, ImageNet Large Scale Visual Recognition Challenge, `https://arxiv.org/pdf/1409.0575.pdf`



Same figure as at the beginning of the session, now that you have the tools to read it. AlexNet is a convolutional network, so it is built out of exactly the operations of this section.
The message is in the title of the block: **nothing conceptually new**. Convolutional networks are LeCun's, from 1989, and backpropagation dates from 1986. What arrived in 2012 was the data (ImageNet, a million labelled images, which took an enormous annotation effort) and the compute, two consumer graphics cards. This answers the question left open by the AI timeline of session 1.

It is worth taking as a lesson rather than as an anecdote: a method can be correct and unusable for twenty-five years, waiting for the data and the hardware to arrive.

Outline

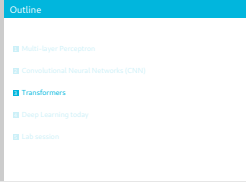
1 Multi-layer Perceptron

2 Convolutional Neural Networks (CNN)

3 Transformers

4 Deep Learning today

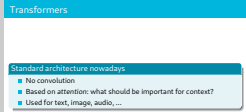
5 Lab session



Transformers

Standard architecture nowadays

- No convolution
- Based on *attention*: what should be important for context?
- Used for text, image, audio, ...



Transformers are today's standard architecture, and they are worth placing with respect to the two you have just seen, using the same question as the rest of the session: what does the architecture assume about the data?
An MLP assumes nothing, and pays for it. A CNN assumes a regular grid, with patterns that are local and independent of position, and wins on images. A Transformer assumes something much weaker: that the input is a *set of elements that interact*, without saying which ones. Instead of being given a fixed neighbourhood, it *learns* which elements matter for which. The central element in that respect is *attention*.

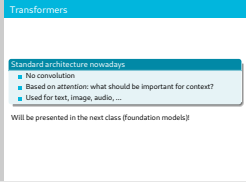
That weaker assumption is what makes it usable on almost any kind of data, and also what makes it more data-hungry than a CNN on images. How attention actually works, and how to use pre-trained Transformers, is the subject of session 5.

Transformers

Standard architecture nowadays

- No convolution
- Based on *attention*: what should be important for context?
- Used for text, image, audio, ...

Will be presented in the next class (foundation models)!



Transformers are today's standard architecture, and they are worth placing with respect to the two you have just seen, using the same question as the rest of the session: what does the architecture assume about the data?
An MLP assumes nothing, and pays for it. A CNN assumes a regular grid, with patterns that are local and independent of position, and wins on images. A Transformer assumes something much weaker: that the input is a *set of elements that interact*, without saying which ones. Instead of being given a fixed neighbourhood, it *learns* which elements matter for which. The central element in that respect is *attention*.

That weaker assumption is what makes it usable on almost any kind of data, and also what makes it more data-hungry than a CNN on images. How attention actually works, and how to use pre-trained Transformers, is the subject of session 5.

Outline

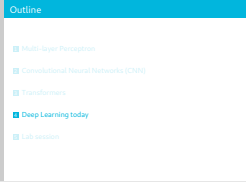
1 Multi-layer Perceptron

2 Convolutional Neural Networks (CNN)

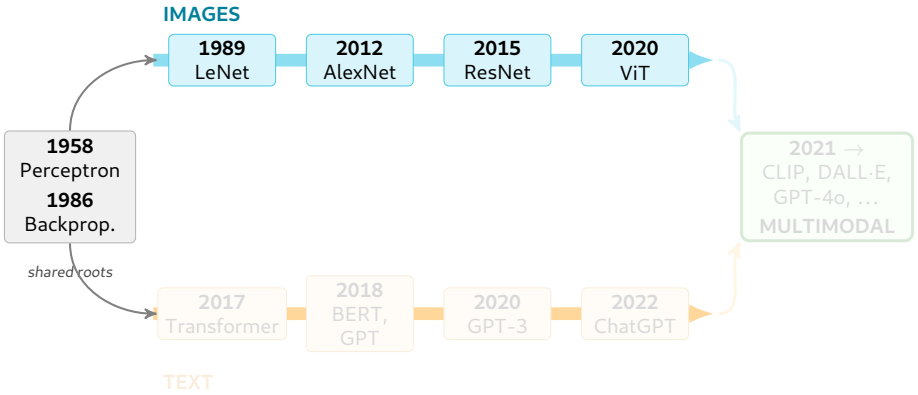
3 Transformers

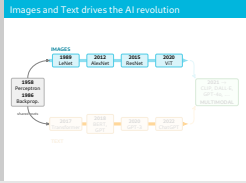
4 Deep Learning today

5 Lab session



Images and Text drives the AI revolution

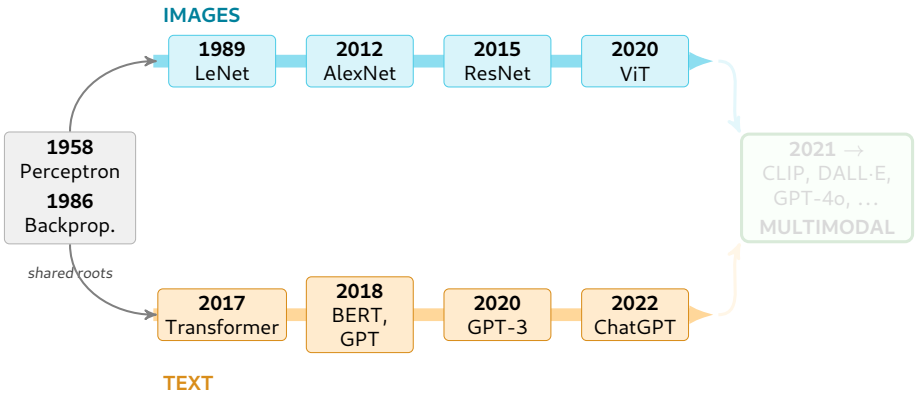


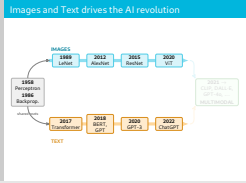


A timeline regrouping the most important developments in deep learning in the last decades.
It started with image processing, with a breaktrhough in 2012.
Now, the most recent developments (and hype) are mostly centered around text processing. The first major breakthrough in text processing was the Transformer in 2017.
After 2020, the two branches of deep learning are converging: Transformers are now used for images as well.
Most recent models are multimodal: they are capable of handling both images and text using the same representations! This is major asset, because knowledge may now be transferred from one domain to another.

Note, again, the shared roots on the left: both branches descend from the same perceptron and the same backpropagation.

Images and Text drives the AI revolution

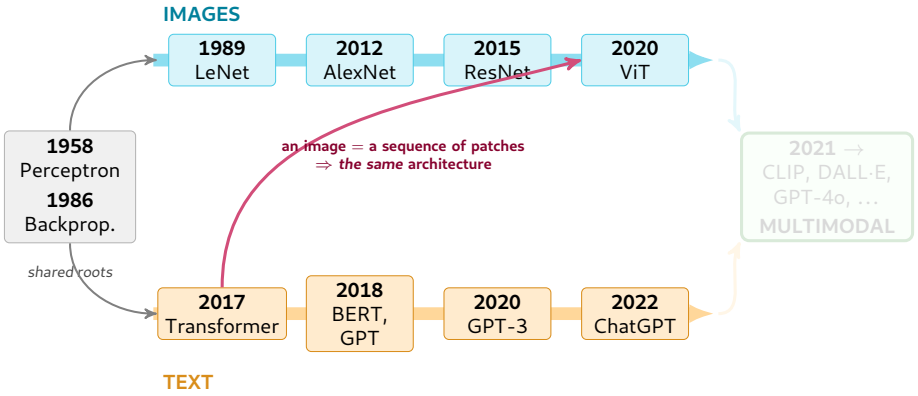




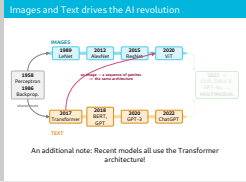
A timeline regrouping the most important developments in deep learning in the last decades.
It started with image processing, with a breaktrhough in 2012.
Now, the most recent developments (and hype) are mostly centered around text processing. The first major breakthrough in text processing was the Transformer in 2017.
After 2020, the two branches of deep learning are converging: Transformers are now used for images as well.
Most recent models are multimodal: they are capable of handling both images and text using the same representations! This is major asset, because knowledge may now be transferred from one domain to another.

Note, again, the shared roots on the left: both branches descend from the same perceptron and the same backpropagation.

Images and Text drives the AI revolution



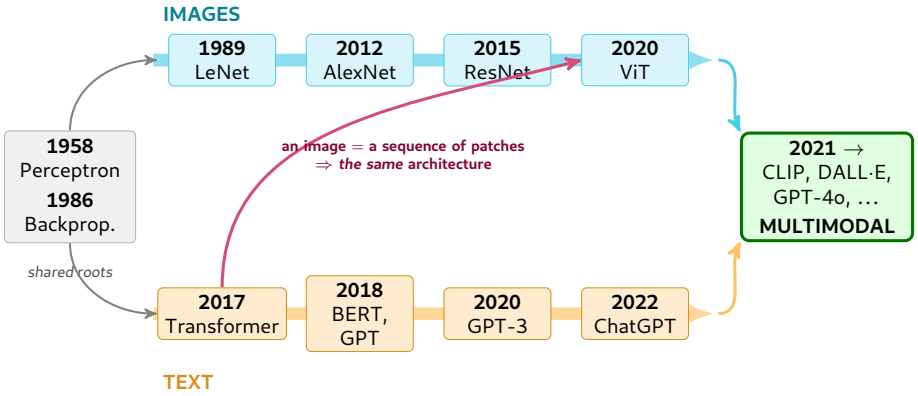
An additional note: Recent models all use the Transformer architecture!



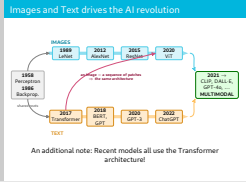
A timeline regrouping the most important developments in deep learning in the last decades.
It started with image processing, with a breaktrhough in 2012.
Now, the most recent developments (and hype) are mostly centered around text processing. The first major breakthrough in text processing was the Transformer in 2017.
After 2020, the two branches of deep learning are converging: Transformers are now used for images as well.
Most recent models are multimodal: they are capable of handling both images and text using the same representations! This is major asset, because knowledge may now be transferred from one domain to another.

Note, again, the shared roots on the left: both branches descend from the same perceptron and the same backpropagation.

Images and Text drives the AI revolution



An additional note: Recent models all use the Transformer architecture!



A timeline regrouping the most important developments in deep learning in the last decades.
It started with image processing, with a breaktrhough in 2012.
Now, the most recent developments (and hype) are mostly centered around text processing. The first major breakthrough in text processing was the Transformer in 2017.
After 2020, the two branches of deep learning are converging: Transformers are now used for images as well.
Most recent models are multimodal: they are capable of handling both images and text using the same representations! This is major asset, because knowledge may now be transferred from one domain to another.

Note, again, the shared roots on the left: both branches descend from the same perceptron and the same backpropagation.

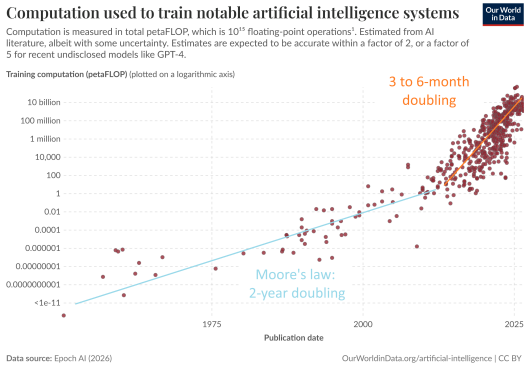
Models keep growing in size

Number of parameters

- AlexNet (2012): 62 **Million**
- GPT-3 (2020): 175 **Billion**

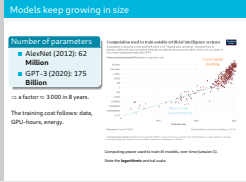
$\Rightarrow$ a factor $\approx$ 3 000 in 8 years.

The training cost follows: data, GPU-hours, energy.



Computing power used to train AI models, over time (session 1).

Note the **logarithmic** vertical scale.



Two numbers, eight years apart, and a factor of three thousand between them. The figure shows the growth of computing power used for training AI models over time. The scale: the vertical axis is the computing power used for training, and it is **logarithmic**, so a straight line means exponential growth.

It is worth translating into concrete terms. 175 billion parameters stored in 16 bits is about 350 GB of weights: that does not fit on any GPU you are likely to own. A single training run of that size costs millions of euros and emits thousands of tonnes of CO_2, and only a handful of organizations in the world can afford one.

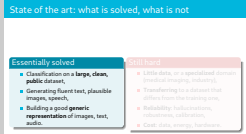
State of the art: what is solved, what is not

Essentially solved

- Classification on a **large, clean, public** dataset,
- Generating fluent text, plausible images, speech,
- Building a good **generic representation** of images, text, audio.

Still hard

- **Little data**, or a **specialized** domain (medical imaging, industry),
- **Transferring** to a dataset that differs from the training one,
- **Reliability**: hallucinations, robustness, calibration,
- **Cost**: data, energy, hardware.



The left column is what you observed on commercial models (e.g., ChatGPT) and is, from a research point of view, mostly settled. The right column is what engineers and researchers are developing. The second item is the least intuitive one and the most common in practice: a model at 99% on its benchmark can collapse on data collected by another hospital, another sensor, or another year. A score is attached to a data distribution, not to a task.

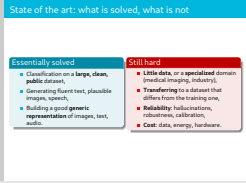
State of the art: what is solved, what is not

Essentially solved

- Classification on a **large, clean, public** dataset,
- Generating fluent text, plausible images, speech,
- Building a good **generic representation** of images, text, audio.

Still hard

- **Little data**, or a **specialized** domain (medical imaging, industry),
- **Transferring** to a dataset that differs from the training one,
- **Reliability**: hallucinations, robustness, calibration,
- **Cost**: data, energy, hardware.

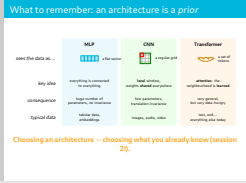


The left column is what you observed on commercial models (e.g., ChatGPT) and is, from a research point of view, mostly settled. The right column is what engineers and researchers are developing. The second item is the least intuitive one and the most common in practice: a model at 99% on its benchmark can collapse on data collected by another hospital, another sensor, or another year. A score is attached to a data distribution, not to a task.

What to remember: an architecture is a *prior*

	MLP	CNN	Transformer
sees the data as…	a flat vector	a regular grid	a set of tokens
key idea	everything is connected to everything	**local** window, weights **shared** everywhere	**attention**: the neighbourhood is **learned**
consequence	huge number of parameters, no invariance	few parameters, translation invariance	very general, but very data-hungry
typical data	tabular data, embeddings	images, audio, video	text, and… everything else today

Choosing an architecture = choosing what you already know (session 2!).



The recap of the architectures. It is easier to read by rows than by columns. The three architectures differ in **one** thing: what they take for granted about the input. The MLP takes nothing for granted, so it has to learn everything from examples. The CNN is given locality and translation, so it needs far fewer parameters and far less data; in exchange it is irrelevant for data with no grid structure. The Transformer assumes only that the elements interact, and learns *which* ones do; that generality is paid for in data.

This is the "priors and constraints" slide of session 2, one level up: there the prior was in the family of functions (e.g., a straight boundary, a tree, a margin) and here it is wired into the architecture itself. Same idea, same practical consequence: with little data, prefer a model that already knows something about your problem.

A useful question for your project: which of these three columns does *my* data belong to: tabular features or embeddings, a regular grid, or a sequence of interacting elements? That single question rules out most of the wrong choices.

Conclusion

Conclusion

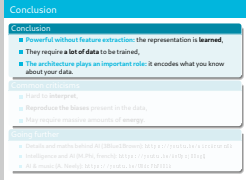
- **Powerful without feature extraction:** the representation is **learned**,
- They require **a lot of data** to be trained,
- **The architecture plays an important role:** it encodes what you know about your data.

Common criticisms

- Hard to **interpret**,
- **Reproduce the biases** present in the data,
- May require massive amounts of **energy**.

Going further

- Details and maths behind AI (3Blue1Brown): `https://youtu.be/aircAruvnKk`
- Intelligence and AI (M.Phi, french): `https://youtu.be/AwUpxjODogQ`
- AI & music (A. Neely): `https://youtu.be/U8dcFhF0Dlk`



Let's wrap-up: Deep learning learns everything from data. It removes the need to design features by hand, but not the need for information: what a domain expert used to supply as knowledge, the model now has to extract from examples. With a thousand samples you are often better served by the `sklearn` models of sessions 2 and 3.

Deep learning has three main limitations:

Interpretability: the model is a black box, a real problem in medicine, justice or credit scoring.

Bias: a model reproduces, and often amplifies, what is in its training data (a statistical property of the method, not a moral failing of the algorithm, which is why good intentions do not fix it).

Cost: energy, hardware, and a concentration of capability in very few hands.

None of this is a reason not to use these tools; all of it is a reason to be able to say what your model does, on which data, and what happens when it is wrong.

Conclusion

Conclusion

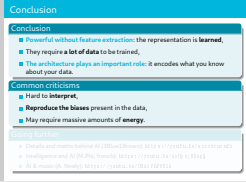
- **Powerful without feature extraction:** the representation is **learned**,
- They require **a lot of data** to be trained,
- **The architecture plays an important role:** it encodes what you know about your data.

Common criticisms

- Hard to **interpret**,
- **Reproduce the biases** present in the data,
- May require massive amounts of **energy**.

Going further

- Details and maths behind AI (3Blue1Brown): `https://youtu.be/aircAruvnKk`
- Intelligence and AI (M.Phi, french): `https://youtu.be/AwUpxjODogQ`
- AI & music (A. Neely): `https://youtu.be/U8dcFhF0Dlk`



Let's wrap-up: Deep learning learns everything from data. It removes the need to design features by hand, but not the need for information: what a domain expert used to supply as knowledge, the model now has to extract from examples. With a thousand samples you are often better served by the `sklearn` models of sessions 2 and 3.

Deep learning has three main limitations:

Interpretability: the model is a black box, a real problem in medicine, justice or credit scoring.

Bias: a model reproduces, and often amplifies, what is in its training data (a statistical property of the method, not a moral failing of the algorithm, which is why good intentions do not fix it).

Cost: energy, hardware, and a concentration of capability in very few hands.

None of this is a reason not to use these tools; all of it is a reason to be able to say what your model does, on which data, and what happens when it is wrong.

Conclusion

Conclusion

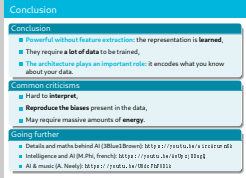
- **Powerful without feature extraction:** the representation is **learned**,
- They require **a lot of data** to be trained,
- **The architecture plays an important role:** it encodes what you know about your data.

Common criticisms

- Hard to **interpret**,
- **Reproduce the biases** present in the data,
- May require massive amounts of **energy**.

Going further

- Details and maths behind AI (3Blue1Brown): `https://youtu.be/aircAruvnKk`
- Intelligence and AI (M.Phi, french): `https://youtu.be/AwUpxjODogQ`
- AI & music (A. Neely): `https://youtu.be/U8dcFhF0Dlk`



Let's wrap-up: Deep learning learns everything from data. It removes the need to design features by hand, but not the need for information: what a domain expert used to supply as knowledge, the model now has to extract from examples. With a thousand samples you are often better served by the `sklearn` models of sessions 2 and 3.

Deep learning has three main limitations:

Interpretability: the model is a black box, a real problem in medicine, justice or credit scoring.

Bias: a model reproduces, and often amplifies, what is in its training data (a statistical property of the method, not a moral failing of the algorithm, which is why good intentions do not fix it).

Cost: energy, hardware, and a concentration of capability in very few hands.

None of this is a reason not to use these tools; all of it is a reason to be able to say what your model does, on which data, and what happens when it is wrong.

Outline

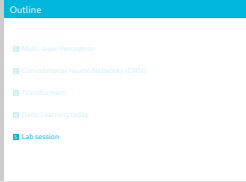
1 Multi-layer Perceptron

2 Convolutional Neural Networks (CNN)

3 Transformers

4 Deep Learning today

5 Lab session



Lab Session 4: Deep Learning with PyTorch

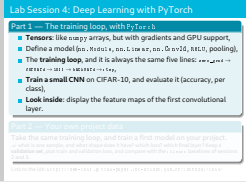
Part 1 — The training loop, with `PyTorch`

- **Tensors**: like `numpy` arrays, but with gradients and GPU support,
- Define a model (`nn.Module`, `nn.Linear`, `nn.Conv2d`, `ReLU`, pooling),
- The **training loop**, and it is always the same five lines: `zero_grad` → `forward` → `loss` → `backward` → `step`,
- **Train a small CNN** on CIFAR-10, and evaluate it (accuracy, per class),
- **Look inside**: display the feature maps of the first convolutional layer.

Part 2 — Your own project data

Take the same training loop, and train a first model on your project.
⇒ what is one sample, and what shape does it have? which loss? which final layer? Keep a **validation set**, plot train *and* validation loss, and compare with the `sklearn` baselines of sessions 2 and 3.

Link to the lab: `https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab4/`



Part 1 is a tutorial, and it has one objective: that the **training loop** becomes familiar. Those five lines are generally identical for every model, every modality and every loss, from the small network of the lab up to a language model.
Be careful: Don't forget `optimizer.zero_grad()`, since gradients accumulate, and don't forget `torch.no_grad()` at test time.
Part 2 is the part that matters for your project, and the first question is not "which architecture" but "what is one sample, and what shape does it have".
Compare your result with the `sklearn` models of the previous sessions: with a thousand samples, a small neural network very often *loses* against a well-tuned classical model. Finding that out is a perfectly good result, and it is the best possible motivation for session 5 on pre-trained models.

Two reminders: train on a small subsample before launching anything long, and keep your results; they may be useful for the final presentation!

Lab Session 4: Deep Learning with PyTorch

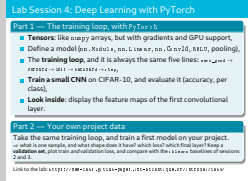
Part 1 — The training loop, with `PyTorch`

- **Tensors**: like `numpy` arrays, but with gradients and GPU support,
- Define a model (`nn.Module`, `nn.Linear`, `nn.Conv2d`, `ReLU`, pooling),
- The **training loop**, and it is always the same five lines: `zero_grad` → `forward` → `loss` → `backward` → `step`,
- **Train a small CNN** on CIFAR-10, and evaluate it (accuracy, per class),
- **Look inside**: display the feature maps of the first convolutional layer.

Part 2 — Your own project data

Take the same training loop, and train a first model on your project.
⇒ what is one sample, and what shape does it have? which loss? which final layer? Keep a **validation set**, plot train *and* validation loss, and compare with the `sklearn` baselines of sessions 2 and 3.

Link to the lab: `https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab4/`



Part 1 is a tutorial, and it has one objective: that the **training loop** becomes familiar. Those five lines are generally identical for every model, every modality and every loss, from the small network of the lab up to a language model.
Be careful: Don't forget `optimizer.zero_grad()`, since gradients accumulate, and don't forget `torch.no_grad()` at test time.
Part 2 is the part that matters for your project, and the first question is not "which architecture" but "what is one sample, and what shape does it have".
Compare your result with the `sklearn` models of the previous sessions: with a thousand samples, a small neural network very often *loses* against a well-tuned classical model. Finding that out is a perfectly good result, and it is the best possible motivation for session 5 on pre-trained models.

Two reminders: train on a small subsample before launching anything long, and keep your results; they may be useful for the final presentation!